Transformers

Dhruv Kumar

BITS Pilani

What is a Transformer?

This lecture focuses on **decoder-only Transformers** — the architecture behind GPT, LLaMA, Claude, and almost every modern language model.

> ONE-SENTENCE DEFINITION
>
> A decoder-only Transformer is a stack of blocks in which **attention moves information between token positions** and **MLPs process information within a position**, all writing additively into a shared residual stream, with a **causal mask** that forbids reading the future.

The training objective

We want to predict the probability of the next token given all previous tokens.

$$P_\theta(x_1, \ldots, x_n) = \prod_{t=1}^{n} P_\theta(x_t \mid x_{<t})$$

$$\mathrm{L} = -\frac{1}{n} \sum_{t=1}^{n} \log \mathrm{softmax}(z_t)_{x_t}$$

$$\mathrm{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{|V|} e^{z_j}}$$

- $x_1, \ldots, x_n$ — the sequence of n tokens; x_t — the token at position t; $x_{<t}$ — every token before position t.
- z_t — the model's output scores (logits) at position t, one number per vocabulary token, shape $|V|$.
- $\mathrm{softmax}(z_t)$ — turns the $|V|$ logits into a probability distribution over the vocabulary: exponentiate each score, then divide by the sum over all $|V|$ scores, so every entry is non-negative and all entries sum to 1.
- $\mathrm{softmax}(z_t)_{x_t}$ — the probability the model assigned to the actual next token; L is the average negative log of that probability over the sequence.

Tokens → embeddings

Text is first split into tokens. *(Self-study: byte-pair encoding is the standard scheme used to do this split.)*

$$W_E \in \mathrm{R}^{|V| \times d}, \qquad X = \mathrm{lookup}(W_E) \in \mathrm{R}^{n \times d}$$

Each token id selects one row of the learned embedding matrix W_E. Stacking a sequence's rows gives X.

- $|V|$ — vocabulary size (number of distinct tokens)
- d — embedding dimension (width of each token's vector)
- n — sequence length (number of tokens)

HOW IS W_E OBTAINED?

W_E is **not** trained separately. It starts as random numbers and is learned end-to-end, jointly with every other weight in the network, by backpropagating the same next-token loss from the previous slide.

The residual stream

Single token (vector form) — $x \in \mathrm{R}^d$ is one token's representation:

$$x \leftarrow x + \mathrm{Attn}(\mathrm{norm}(x)), \qquad x \leftarrow x + \mathrm{MLP}(\mathrm{norm}(x))$$

Whole sequence (matrix form) — $X \in \mathrm{R}^{n \times d}$ updates every token at once:

$$X \leftarrow X + \mathrm{Attn}(\mathrm{norm}(X)), \qquad X \leftarrow X + \mathrm{MLP}(\mathrm{norm}(X))$$

Every sub-layer reads a normalized copy of the stream and **adds** its result back in. The final representation is the embedding *plus* the sum of every sub-layer's contribution.

DIVISION OF LABOUR

Attention moves information *between* lanes (token positions).
MLPs process information *within* a lane.

Positional information

Self-attention treats the input as a set of tokens — by itself, it cannot tell that token 3 comes after token 2. Position has to be injected explicitly.

$$X \leftarrow X + P, \qquad P \in \mathrm{R}^{n \times d}$$

The simplest scheme: a positional encoding matrix P has one row per position, and row t is added directly to token t's embedding — same shape as X, so it adds in cleanly.

> SELF-STUDY
>
> **RoPE** (rotary position embeddings) is the scheme used by most modern models (LLaMA, Mistral, and others).

Query, Key, Value

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

$X \in \mathrm{R}^{n \times d}$ — input token sequence matrix. $W_Q, W_K \in \mathrm{R}^{d \times d_k}$ and $W_V \in \mathrm{R}^{d \times d_v}$ — learned projection matrices, giving $Q, K \in \mathrm{R}^{n \times d_k}$ and $V \in \mathrm{R}^{n \times d_v}$.

Query

What this token is **looking for**.

Key

What this token **advertises** about itself.

Value

What this token **hands over** if it is attended to.

The similarity between a query and a key is simply a **dot product**.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

1. **Scores** $S = QK^\top/\sqrt{d_k}$, shape $n \times n$ — how much each token's query matches every token's key.
2. **Weights** $A = \text{softmax}(S)$, shape $n \times n$ — each row turned into a probability distribution; every row sums to 1.
3. **Output** $O = AV$, shape $n \times d_v$ — each token's output is a weighted average of all value vectors.

> WHY DIVIDE BY √D_K
>
> Purely for better training: it keeps the scores in a healthy range so the softmax that follows does not saturate.

The causal mask

$$M_{ij} = \begin{cases} 0 & j \le i \\ -\infty & j > i \end{cases} \qquad M \in \mathrm{R}^{n \times n}$$

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)$$

Position i may only look at positions $j \le i$ — itself and the past. Adding $-\infty$ before the softmax drives those entries to exactly zero once the row is renormalized, so a prediction at position t never sees position $t+1$ onward.

What the mask looks like

q\k	0	1	2	3	4	5	6	7
0	.00	–	–	–	–	–	–	–
1	.50	.50	–	–	–	–	–	–
2	.33	.33	.33	–	–	–	–	–
3	.25	.25	.25	.25	–	–	–	–
4	.20	.20	.20	.20	.20	–	–	–
5	.17	.17	.17	.17	.17	.17	–	–
6	.14	.14	.14	.14	.14	.14	.14	–
7	.13	.13	.13	.13	.13	.13	.13	.13

Rows = queries, columns = keys. Shaded cells are allowed ($j \leq i$); dashed cells are blocked ($j > i$).

Worked example — from X to Q, K, V

3 tokens ($n = 3$), embedding width $d = 3$. **Single head**, so $d_k = d_v = d = 3$ — as in a real model, this keeps the attention output the same shape as X.

$$X = \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

$$W_Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix} \quad W_K = \begin{bmatrix} 1 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} \quad W_V = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & -1 & 1 \end{bmatrix}$$

One entry, worked out — Q_{11} is row 1 of X dotted with column 1 of W_Q: $Q_{11} = X_{11}W^Q_{11} + X_{12}W^Q_{21} + X_{13}W^Q_{31} = 1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 = 2$

Every other entry of $Q = XW_Q$, $K = XW_K$, $V = XW_V$ follows the same row-times-column pattern:

$$Q = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 0 \end{bmatrix} \qquad K = \begin{bmatrix} 1 & 2 & 1 \\ 1 & 1 & 2 \\ 2 & 1 & 1 \end{bmatrix} \qquad V = \begin{bmatrix} 2 & -1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

Worked example — scores to output

Scores $QK^\top$, then scaled by $1/\sqrt{d_k} = 1/\sqrt{3} \approx 0.5774$:

$$QK^\top = \begin{bmatrix} 5 & 5 & 6 \\ 6 & 5 & 5 \\ 3 & 2 & 3 \end{bmatrix} \qquad \frac{QK^\top}{\sqrt{d_k}} = \begin{bmatrix} 2.89 & 2.89 & 3.46 \\ 3.46 & 2.89 & 2.89 \\ 1.73 & 1.15 & 1.73 \end{bmatrix}$$

Causal mask applied ($j > i$ blocked), then row-wise softmax:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) = \begin{bmatrix} 1.000 & 0 & 0 \\ 0.640 & 0.360 & 0 \\ 0.390 & 0.219 & 0.390 \end{bmatrix}$$

Output $O = AV$ — row 3's output is a weighted blend of v_1, v_2, v_3 using weights $0.39,\ 0.22,\ 0.39$:

$$O = AV = \begin{bmatrix} 2.000 & -1.000 & 1.000 \\ 1.640 & -0.640 & 1.000 \\ 1.390 & 0.000 & 0.610 \end{bmatrix}$$

O has shape 3×3 — exactly X's shape. Because this is a single head with $d_v = d$, O can be added straight into the residual stream: no output projection needed.

Multi-head attention

A single attention head must compromise across every reason a token might attend elsewhere. Running h heads in parallel lets each one learn a **different kind of relationship**, each with its own probability budget.

$$\text{head}_i = \text{Attention}\left(XW_Q^{(i)},\ XW_K^{(i)},\ XW_V^{(i)}\right)$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\, W_O$$

Models pick h and d_v so that $h \times d_v = d$, keeping the concatenated heads at width d — matching the residual stream **before** $W_O \in \mathrm{R}^{d \times d}$ is even applied. Each head only sees its own d_v-wide slice, so W_O's job is to **mix information across heads**.

1. X: shape $n \times d$, split into h heads of width $d_k = d_v = d/h$.
2. Each head runs attention independently → output $n \times d_v$ per head.
3. Concatenate all h outputs → already back to $n \times d$.
4. Multiply by W_O to mix across heads before writing into the residual stream.

> **WHY THE EARLIER WORKED EXAMPLE SKIPPED W_O**
>
> Single head ($h = 1$), $d_v = d$: concatenation is a no-op, so W_O would just repeat what W_V already does.

Normalization

$$\mathrm{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

$x \in \mathrm{R}^d$ — one token's vector; μ, σ^2 — mean and variance computed across the d features of that **one token** (not across the batch); $\gamma, \beta \in \mathrm{R}^d$ — learned scale and shift.

Normalizing each token's vector before it enters attention or the MLP keeps activations in a stable range, which keeps training stable — especially in deep stacks.

TWO VARIANTS, SAME IDEA

LayerNorm (above) and **RMSNorm** are essentially the same operation; RMSNorm simply skips mean-centering: $\mathrm{RMSNorm}(x) = \gamma \odot x / \sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}$.

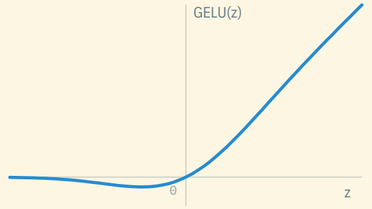
The feed-forward network (MLP)

$$\mathrm{FFN}(x) = W_2\,\phi(W_1 x + b_1) + b_2$$

$x \in \mathrm{R}^d$; $W_1 \in \mathrm{R}^{d_{ff} \times d}$, $b_1 \in \mathrm{R}^{d_{ff}}$; ϕ — a nonlinearity, applied elementwise; $W_2 \in \mathrm{R}^{d \times d_{ff}}$, $b_2 \in \mathrm{R}^d$; typically $d_{ff} = 4d$.

$$\phi(z) = \mathrm{GELU}(z) = z\,\Phi(z)$$

The most widely used choice is **GELU**, where $\Phi(z)$ is the standard normal cumulative distribution function — unlike ReLU, it does not sharply zero out negative inputs, only smoothly damps them.



Applied identically and independently to **every** token position — unlike attention, there is no mixing across positions here.

> HIGH-LEVEL INTUITION
>
> Attention decides **what information to gather**; the MLP is where the model does nonlinear processing of that information, and appears to store much of what it "knows".

The complete decoder block

$$h = x + \mathrm{MHA}\left(\mathrm{norm}(x),\ \text{causal mask}\right)$$

$$y = h + \mathrm{FFN}\left(\mathrm{norm}(h)\right)$$

Stack this block L times — that is the whole architecture.

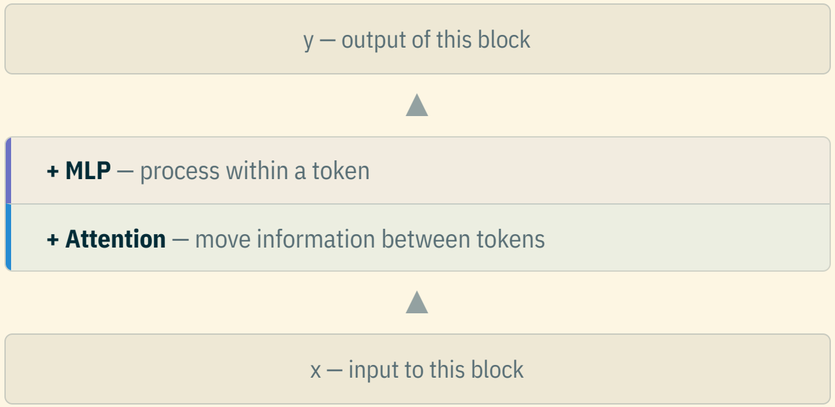
#	OPERATION	SHAPE OUT
1	Token ids	`n`
2	Embedding lookup W_E	`n × d`
3	**× L:** norm → causal MHA → residual add	`n × d`
4	**× L:** norm → FFN → residual add	`n × d`
5	Final norm, then unembed W_U	`n × |V|`
6	Cross-entropy loss vs. next token x_{t+1}	scalar

Row t of the logits is built from $x_1, \ldots, x_t$ (the causal mask lets it see up to and including x_t), so it predicts the *next* token, x_{t+1} — training compares row t against x_{t+1} for every t, i.e. the target sequence is just the input shifted one position to the left.

Inside step 3, tensors reshape `n×d → h×n×d_k` so the $QK^\top$ matmul batches over all h heads at once, then merge back before W_O.

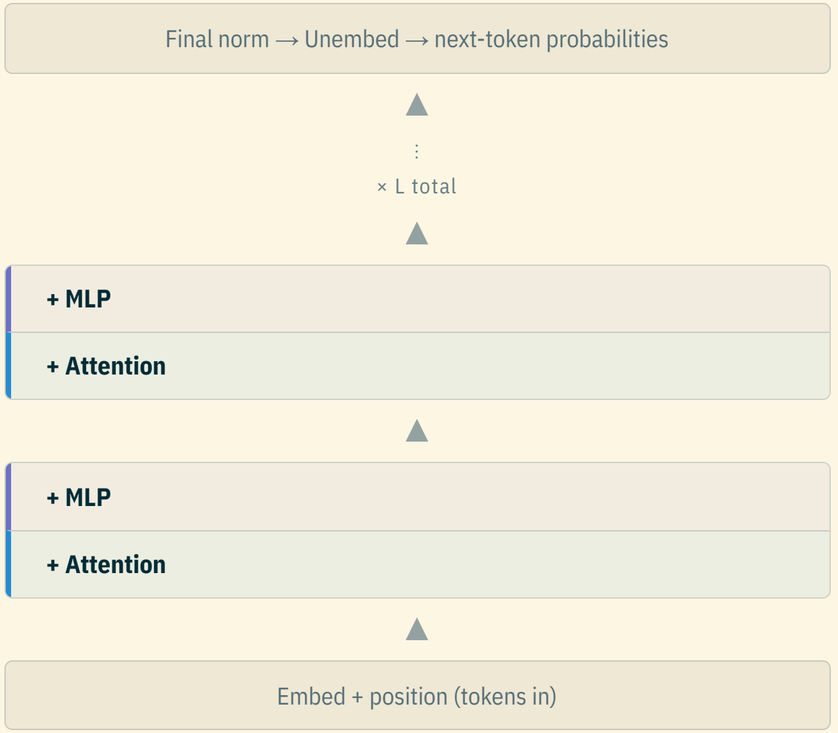
One block: attention + MLP

Every block does exactly two things to the residual stream, one after another.



This is **one decoder block** — the unit that gets stacked L times to build the whole model.

Stack it × L



Same block, repeated L times. Every block reads from and writes back into the **same residual stream** — depth is just stacking this one unit again and again.

Unembedding: logits → next token

After the last block, $H \in \mathrm{R}^{n \times d}$ is normalized, then projected out of the residual stream and back into vocabulary space:

$$Z = \mathrm{norm}(H)\, W_U, \qquad W_U \in \mathrm{R}^{d \times |V|}, \qquad Z \in \mathrm{R}^{n \times |V|}$$

W_U is the mirror image of the embedding matrix W_E from the "Tokens → embeddings" slide — instead of looking up a row for a token id, it turns a d-dimensional vector back into $|V|$ scores. Row t of Z, written $z_t \in \mathrm{R}^{|V|}$, is built from $x_1, \ldots, x_t$ — so it holds one raw score per vocabulary token for predicting the **next** token, x_{t+1}, not x_t itself.

$\mathrm{softmax}(z_t)$ turns those scores into a probability distribution over the vocabulary. How that distribution becomes an actual next token depends on what you're doing:

- **Training** — compare $\mathrm{softmax}(z_t)$ to the true next token x_{t+1} with the cross-entropy loss; every position is scored at once.
- **Generation** — only the last position's z_n is used: either take the highest-probability token (*greedy*), or sample a token from $\mathrm{softmax}(z_n)$ (*stochastic decoding*). The chosen token is appended to the sequence, and the whole model runs again to produce the next one.

Reading list

START HERE

Vaswani et al., "Attention Is All You Need" (2017)
The Annotated Transformer — Harvard NLP
The Illustrated Transformer — Jay Alammar

WATCH & RUN

Karpathy, "Let's build GPT: from scratch, in code, spelled out"
nanoGPT — Karpathy

FORMAL REFERENCE

Phuong & Hutter, "Formal Algorithms for Transformers" (2022)

SELF-STUDY: POSITIONAL ENCODING

Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021)